London Equation: Simulation Techniques for Magnetic-Field Penetration (Q6)

This document supports plan items #6.1 through #6.5 of `Plan.md` and is the deliverable for issue #1. It records the candidate simulation techniques and the reason for each choice. It does not change source code. The planned modules will use these techniques.

1. Purpose and scope

A London-equation simulation must show how a static magnetic field decays inside a superconductor. It must produce the field magnitude $B(x)$ against position x. It must expose the penetration depth λ as the length over which B falls from the boundary value toward zero. It must also show the temperature dependence of λ and the contrast between the slab and cylinder geometries. Every technique below must reproduce the same analytic answer. The report covers direct evaluation, finite differences, relaxation, and shooting.

2. Physics recap

2.1 The London equations

The London brothers replaced the current density J of the superconducting state with two equations [1]:

$$\frac{dJ}{dt} = \frac{n_s e^2}{m}, E \qquad \nabla \times J = -\frac{n_s e^2}{m}, B$$

The first equation says that a steady current flows without an electric field. The second equation ties $\nabla \times J$ to B. Ampere's law $\nabla \times B = \mu_0 J$ then combines with the second equation into $\nabla^2 B = B/\lambda^2$:

$$\lambda = \sqrt{\frac{m}{\mu_0 n_s e^2}}$$

The expression gives λ in meters for n_s in m^{-3}. With n_s near one carrier per atom, λ is of order 20 to $40, \mathrm{nm}$ for conventional superconductors. Tinkham lists $16, \mathrm{nm}$ as the London estimate for aluminum and about $50, \mathrm{nm}$ as the measured value [2]. The penetration depth is a mesoscopic length, far below the size of a visible sample.

2.2 The Meissner effect and the 1D slab solution

The Meissner effect is the expulsion of magnetic flux at the transition to the superconducting state [3]. That expulsion happens on any path through the phase diagram. A perfect conductor would

WebsitePublisher.ai Your AI Builds Websites. You Just Talk. websitepublisher.ai →

freeze the flux in place instead. A superconductor produces a screening supercurrent in a boundary layer of thickness λ. The interior field is zero beyond that layer. This screening is the effect that the simulation must show.

Consider a slab of thickness d that fills -d/2 < x < d/2. An applied field B_0 runs along the slab faces. For a wide slab, the demagnetization effect is negligible. The field at each face then keeps the value B_0. The boundary conditions read $B(\pm d/2) = B_0$. The 1D form of $\lambda^2 \nabla^2 B = B$ is $\lambda^2, d^2B/dx^2 = B$:

$$B(x) = B_0, \frac{\cosh(x/\lambda)}{\cosh(d/2\lambda)}$$

The coordinate origin $x = 0$ is at the center of the slab. At each face the solution equals B_0. At the center it equals $B_0 / \cosh(d/2\lambda)$. For $d \gg \lambda$ the center is field-free, and the decay constant near each face is λ. The same derivation appears in Kittel [7] and in Poole et al. [8]. The two-fluid model of Gorter and Casimir gives the law that item #6.4 uses [4]:

$$\lambda(T) = \frac{\lambda_0}{\sqrt{1 - (T/T_c)^4}}$$

The law diverges at $T = T_c$. The simulator must keep T/T_c below 1. λ_0 is the penetration depth at $T = 0$.

WebsitePublisher.ai Your AI Builds Websites. You Just Talk. websitepublisher.ai →

Symbol	Definition	Value or unit
J	supercurrent density	$\mathrm{A}, \mathrm{m}^{-2}$
E	electric field	$\mathrm{V}, \mathrm{m}^{-1}$
n_s	superconducting carrier density	m^{-3}
e	elementary charge	$1.602,176,634 \times 10^{-19}, \mathrm{C}$
m	electron mass	$9.109,383,7 \times 10^{-31}, \mathrm{kg}$
B	magnetic flux density	T
μ_0	vacuum permeability	$1.256,637,061 \times 10^{-6}, \mathrm{H}, \mathrm{m}^{-1}$
λ	penetration depth	m
d	slab thickness	m
x	position from the slab center	m
B_0	applied flux density	T
T, T_c	temperature and critical temperature	K
λ_0	penetration depth at $T = 0$	m
a, r	cylinder radius and radial coordinate	m
u	slope variable, $u = dB/dx$	T/m
Δx	grid step	m
N	number of grid points	count
ϵ	relaxation tolerance	dimensionless
$\tilde{x}$, $\tilde{B}$, $\tilde{d}$	dimensionless x, B, d	dimensionless
ξ	coherence length	m
H_{c1}, H_c	lower and thermodynamic critical fields	A/m
I_0	modified Bessel function, order 0	dimensionless

2.3 Limits of the local London model

The London model treats J at each point as a local response to E and B. That assumption fails in several known cases. Pippard showed that the cleanest materials have a coherence length ξ above λ [5]. The supercurrent then responds to the field over a finite area. The local model also fails near a face once vortices appear. The vortex state starts above H_{c1} in a type-II superconductor. Near T_c the

WebsitePublisher.ai Your AI Builds Websites. You Just Talk. websitepublisher.ai →

order parameter varies with position. Ginzburg-Landau theory then must replace the fixed n_s of the London model. The ranges in #4.2 respect these limits.

3. Simulation techniques

3.1 Direct analytic evaluation on a grid

The analytic solution is exact and free of discretization error. It is the reference for every numerical method. The module `slab.py` (item #6.2) will evaluate $B(x)/B_0$ on a grid of N points. It will then return the grid with the array:

```python
import numpy as np

def slab_field(x, d, lam, b0):
    """Return B(x) for a slab of thickness d and penetration depth lam.

    x is measured from the slab center, so the faces sit at -d/2 and +d/2.
    lam is lambda(T) from temperature.py, in SI units of m.
    """
    return b0 * np.cosh(x / lam) / np.cosh(d / (2 * lam))
```

The function accepts scalars and arrays, because `np.cosh` broadcasts over the grid. A plot at $d = 10\lambda$ shows a boundary layer of about 3λ at each face.

3.2 Finite differences and the tridiagonal system

The finite-difference method solves the field equation as a boundary value problem. Discretize x into N points with step Δx. At each interior point i, the central difference formula gives

$$\frac{B_{i-1} - 2B_i + B_{i+1}}{\Delta x^2} = \frac{B_i}{\lambda^2}$$

The result is a tridiagonal system of $N - 2$ unknowns. The known face values add to the first and last `rhs` entries. `solve_banded` from SciPy [6] then solves the system in $O(N)$ time:

WebsitePublisher.ai Your AI Builds Websites. You Just Talk. websitepublisher.ai →

```python
from scipy.linalg import solve_banded

def slab_finite_difference(d, lam, b0, n_points):
    """Solve lam^2 d^2B/dx^2 = B for a slab with B(+/- d/2) = b0."""
    x = np.linspace(-d / 2, d / 2, n_points)
    dx = x[1] - x[0]
    main = 2.0 + (dx / lam) ** 2          # (2 + (dx/lam)^2) B_i on the diagonal
    n = n_points - 2                      # unknowns: interior nodes only
    ab = np.zeros((3, n))
    ab[0, 1:] = -1.0                      # superdiagonal: -B_{i+1}
    ab[2, :-1] = -1.0                     # subdiagonal: -B_{i-1}
    ab[1, :] = main
    rhs = np.zeros(n)
    rhs[0] += b0                          # known value at the left face
    rhs[-1] += b0                         # known value at the right face
    return x, np.concatenate(([b0], solve_banded((1, 1), ab, rhs), [b0]))
```

The matrix is symmetric and diagonally dominant, because the diagonal $2 + (\Delta x/\lambda)^2$ exceeds the sum of the two off-diagonal entries. The direct solve is stable for any Δx. That restriction applies to an explicit step for a time-dependent diffusion problem.

3.3 Relaxation as a cross-check

The relaxation method solves the same system by repeated sweeps, without a matrix library. It updates each interior point from its two neighbors:

$$B_i \leftarrow \frac{B_{i-1} + B_{i+1}}{2 + (\Delta x/\lambda)^2}$$

A flat start works, and Jacobi sweeps then converge to the same discrete solution. The sweep repeats until the largest change per sweep drops below $\epsilon = 10^{-10}$:

```python
def slab_relaxation(d, lam, b0, n_points, tol=1e-10, max_sweeps=10000):
    """Relax B[i] <- c*(B[i-1] + B[i+1]), c = 1/(2+(dx/lam)^2)."""
    x = np.linspace(-d / 2, d / 2, n_points)
    b = np.full(n_points, b0)             # start flat at the face value
    c = 1.0 / (2.0 + ((x[1] - x[0]) / lam) ** 2)
    sweeps = 0
    for sweeps in range(max_sweeps):      # Jacobi sweep with a double buffer
        new = b.copy()
        new[1:-1] = c * (b[:-2] + b[2:])
        change = np.abs(new[1:-1] - b[1:-1]).max()
        b = new
        if change < tol:                  # stops at the Jacobi fixpoint
            break
    if change >= tol:                     # the code must report the trip
        raise RuntimeError(f"relaxation hit the {max_sweeps}-sweep cap")
    return x, b
```

WebsitePublisher.ai Your AI Builds Websites. You Just Talk. websitepublisher.ai →

The sweep does not share code with the matrix solve of #3.2. It then can catch an assembly error in that matrix. The sweep count grows as N^2, and the cap keeps the check on a modest grid. The sweep and the solve must agree to within tolerance.

3.4 ODE integration and shooting

The shooting method turns the field equation into an initial value problem. Write the second-order form as a pair of first-order equations, $dB/dx = u$ and $du/dx = B/\lambda^2$. The slope $u(-d/2)$ is unknown. The method integrates from the left face, with that slope as a guess. A root finder then adjusts the guess until the field at the right face equals B_0. `solve_ivp` from SciPy [6] integrates the system:

```
python
from scipy.integrate import solve_ivp
from scipy.optimize import brentq

def rhs(x, y, lam):
    """Right-hand side of dB/dx = u, du/dx = B / lam^2."""
    return [y[1], y[0] / lam ** 2]

def slab_shooting(d, lam, b0, n_points=201, tol=1e-10):
    """Shoot the unknown initial slope u(-d/2) to hit B(+d/2) = b0."""
    def end_field(slope):
        sol = solve_ivp(rhs, (-d / 2, d / 2), (b0, slope), args=(lam,),
                        rtol=tol, atol=tol / 100)
        return sol.y[0, -1] - b0          # residual at the right face

    # the exact slope is -(b0/lam)*tanh(d/2/lam), inside [-b0/lam, b0/lam]
    slope = brentq(end_field, -b0 / lam, b0 / lam)
    x = np.linspace(-d / 2, d / 2, n_points)
    sol = solve_ivp(rhs, (-d / 2, d / 2), (b0, slope), args=(lam,),
                    rtol=tol, atol=tol / 100, t_eval=x)
    return x, sol.y[0]                    # field values on the fixed grid
```

The shooting method does not share an algorithm with the relaxation sweep. Agreement between the two then confirms the finite-difference setup of #3.2.

4. Numerical practice

4.1 Non-dimensionalization

Scale the coordinates before the solver runs. Use $x = \lambda\tilde{x}$ and $B = B_0\tilde{B}$, with tilde marks for the dimensionless quantities. The field equation then reads $d^2\tilde{B}/d\tilde{x}^2 = \tilde{B}$ with $\tilde{B}(\pm\tilde{d}/2) = 1$ and $\tilde{d} = d/\lambda$. The dimensionless form removes hard units from the solvers. It also keeps every result reusable across materials. The modules will work internally in that form and will reattach λ and B_0 at the edges.

WebsitePublisher.ai Your AI Builds Websites. You Just Talk. websitepublisher.ai →

4.2 Mesh resolution and refinement

$B(x)$ changes on the scale of λ near each face. The mesh must resolve that scale. At a step of $\lambda/8$, the relative truncation error is about 3×10^{-4}. At a step of $\lambda/32$, it drops below 10^{-4}. A uniform mesh then puts many points into the field-free core at a large d/λ. The code will use a uniform mesh for small ratios. It will grade the mesh toward the faces for large ratios.

Parameter	Recommended range	Reason
d/λ	0.5 to 100	covers thin films and bulk slabs
points per λ	8 to 32	keeps the max error below 10^{-3}
total points N	101 to 4001	$O(N)$ work per direct solve
T/T_c	0 to 0.99, always below 1	$\lambda(T)$ diverges at T_c
B_0	0.01 to 0.5, T	keep B_0 below H_{c1} or H_c
ϵ	10^{-10}	shared by the three solvers

4.3 Convergence and validation

Each technique must converge to the same answer. The validation driver will compute one numerical answer per technique on one mesh. It will print two error metrics per technique. The metrics are the maximum absolute error and the L^2 error $\|e\|_2 = \sqrt{\sum_i e_i^2, \Delta x}$. The convergence study will halve Δx in steps and log the max error each time. A halved mesh must cut the finite-difference error by about a factor of four. The central formula has error $O(\Delta x^2)$.

N at fixed $d/\lambda = 5$	measured max error
41	2.8×10^{-4}
81	7.0×10^{-5}
161	1.8×10^{-5}

The exact error depends on d/λ. The rows show the factor-of-four trend, which checks the scaling.

4.4 Conditioning and stability

The matrix of #3.2 is diagonally dominant for every Δx and every λ. The direct solve on that matrix is stable. First, `np.cosh` overflows for arguments above about 700. The denominator argument $d/(2\lambda)$ passes that value at a large d/λ. An overflow then turns the finite ratio into `nan`. The code must divide the numerator and the denominator of the ratio by $e^{d/2\lambda}$. Every remaining term is at most 1 after that scaling. Second, the relaxation sweep count grows as N^2, because each sweep moves information by one node. The loop in #3.3 caps the count through `max_sweeps` and must report the

WebsitePublisher.ai Your AI Builds Websites. You Just Talk. websitepublisher.ai →

limit when it trips. Third, the shooting integration amplifies $\exp(x/\lambda)$. The integrator must run with tight tolerances.

5. Techniques in the planned modules

Module	Plan item	Techniques
`slab.py`	#6.2	analytic solution (#3.1) plus finite differences (#3.2)
`cylinder.py`	#6.3	the same methods with modified Bessel functions (I_0, derivative I_1)
`temperature.py`	#6.4	the two-fluid law for $\lambda(T)$ [4] with slider-driven plots
`visualization.py`	#6.5	`imshow` and `contourf` heat maps of the field magnitude
`app.py`	#6.6	interactive inputs for T, B_0, λ, and geometry

`slab.py` will expose one function per technique. Tests then compare the outputs pairwise. `cylinder.py` repeats the same pattern. The analytic answer for a cylinder of radius a with an axial field is $B® = B_0, I_0(r/\lambda)/I_0(a/\lambda)$. The solvers reuse the same discretization. The cylindrical Laplacian adds a B'/r term to the radial form. `temperature.py` will wrap the law of #2.2 in a function. It will pass $\lambda(T)$ over T/T_c in $[0, 0.99]$ to the geometry functions. `visualization.py` will show heat maps of the field magnitude on 2D grids, and `app.py` will connect sliders to those plots. The display then updates on each input change.

6. What was done and why

The body of issue #1 contains the line: “Add comments on what you have done and why.” This document makes those comments at the level of simulation techniques. The answer has three parts. First, it records the physics and the analytic solution of #2. Every numerical step needs that reference answer. Second, it records four techniques in #3 with the numerical practice of #4 beside them. The implementers then do not need to rediscover the mesh, scaling, and stability choices. Third, it maps the techniques onto the planned modules in #5. Reviewers then can check each module of #6.2 through #6.6 against the plan. The document does not change source code. The plan assigns the code work to #6.2 through #6.6.

7. Caveats

The London model holds in a fixed window. It assumes a local response and a static field. The simulators must keep d/λ and the applied field in the ranges of #4.2. They must mark the two-fluid curve as a model law in every plot caption. The failures of #2.3 apply outside the window. For #6.3, the Bessel answer is exact only for an axial field on a cylinder. A transverse field needs a numerical treatment beyond this plan.

WebsitePublisher.ai Your AI Builds Websites. You Just Talk. websitepublisher.ai →

References

1. London, F., London, H. "The electromagnetic equations of the supra-conductor." *Proceedings of the Royal Society A*, 149(866), 71-88, 1935. DOI: `10.1098/rspa.1935.0048`.
2. Tinkham, M. *Introduction to Superconductivity*. 2nd ed. McGraw-Hill, 1996 (Dover reprint, 2004), ch. 1.
3. Meissner, W., Ochsenfeld, R. "Neue Effekte bei Eintritt der Supraleitung." *Naturwissenschaften*, 21, 787-788, 1933.
4. Gorter, C. J., Casimir, H. B. G. "On the theory of superconductivity." *Physica*, 1, 306-320, 1934.
5. Pippard, A. B. "The coherent length of superconductors." *Proceedings of the Royal Society A*, 216, 547-568, 1953.
6. Virtanen, P., Gommers, R., Oliphant, T. E., et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python." *Nature Methods*, 17, 261-272, 2020. DOI: `10.1038/s41592-019-0686-2`.
7. Kittel, C. *Introduction to Solid State Physics*. 8th ed. Wiley, 2005, ch. 10.
8. Poole, C. P., Jr., Farach, H. A., Creswick, R. J., Prozorov, R. *Superconductivity*. 2nd ed. Academic Press (Elsevier), 2014.

WebsitePublisher.ai Your AI Builds Websites. **You Just Talk.** **websitepublisher.ai →**